



AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

A
MINI PROJECT REPORT ON
“The AI, ML, Data Science Salary (2020- 2025)”
FOR

Term Work Examination

***Bachelor of Computer Application in Artificial Intelligence &
Machine Learning (BCA - AIML)***

Year 2025-2026

Ajeenkya DY Patil University, Pune

-Submitted By-

Ms. ISHITA SAWARDEKAR

Under the guidance of

Prof. Vivek More



Ajeenkya DY Patil University

D Y Patil Knowledge City,
Charholi Bk. Via Lohegaon,
Pune - 412105
Maharashtra (India)

Date: 14 / 04 / 2025

CERTIFICATE

This is to certified that Ishita Sawardekar
A student of BCA (AIML) **Sem-IV** URN No 2023-B-13042005
has Successfully Completed the Dashboard Report On

“The AI, ML, Data Science Salary (2020- 2025)”

As per the requirement of
Ajeenkya DY Patil University, Pune was carried out under my
supervision.

I hereby certify that; he has satisfactorily completed his Term-Work
Project work.

Place: Pune

Examiner

Sr.No	Index	Page No
1	Introduction & Objective.	4-5
2	Methodology & Approach	5-7
3	Implementation & Code.	7-14
4	Results & Visualizations.	14-23
5	Conclusion & Future Scope	24-25

Introduction:

The “salaries.csv” dataset provides a detailed overview of compensation trends within the data science and technology industries. It captures a variety of essential attributes including job titles, experience levels, employment types, remote work ratios, company sizes, locations, and actual salaries. This diverse set of features allows for a multifaceted analysis of salary structures across different roles and organizational environments. By leveraging this data, one can uncover how factors such as experience, job role, and remote flexibility influence earnings in the tech workforce. This dataset is particularly valuable for data analysts, HR professionals, and career seekers aiming to understand industry standards and wage disparities. Through visualizations and statistical exploration, it becomes possible to identify patterns, compare job roles, and draw meaningful conclusions about compensation trends in an evolving job market. The dataset thus serves as a practical foundation for salary benchmarking, career planning, and organizational compensation strategies.

Objective:

The primary objective of this analysis is to:

- Explore the distribution of salaries across different experience levels and job titles.
- Visualize trends and patterns using charts like bar plots, box plots, scatter plots, pie charts, and heatmaps.
- Analyze the relationship between remote work, company size, and average salary.
- Identify which roles or conditions lead to higher compensation in the data/tech job market.
- Provide insights to job seekers, employers, and analysts for better decision-making based on real-world salary trends.

Methodology & Approach:

To analyze the salaries.csv dataset effectively, a structured methodology was followed comprising data preparation, exploration, visualization, and interpretation:

1. Data Loading & Cleaning

- Imported the dataset using **pandas**.
- Checked for missing values, duplicates, and inconsistent entries.
- Filtered and standardized categorical data for consistency (e.g., job titles, experience levels).

2. Exploratory Data Analysis (EDA)

- Examined distributions of key variables such as salary, job title, company size, and remote ratio.
- Used statistical summaries to understand central tendencies and variances.

3. Visualization Techniques

- Bar plots to compare average salaries by job title.
- Box plots to observe salary distributions by experience level.
- Scatter plots to explore relationships (e.g., salary vs. remote ratio).
- Pie charts to show categorical distributions like company size.
- Heatmaps to analyze correlations between numerical variables.

4. Interpretation & Insights

- Derived meaningful insights from the visualizations and trends.
- Identified factors significantly influencing salary variations.

Implementation & Code:

STEP 1: Import Required Libraries

- **pandas**: for data manipulation
- **matplotlib** & **seaborn**: for visualizations

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

STEP 2: Load the CSV File

```
# 1. Load the CSV file
df = pd.read_csv('/content/salaries.csv')
```

Reads the dataset into a DataFrame.

Make sure the file exists at that path. For Colab, it should be uploaded to **/content**.

STEP 3: Clean the Data

```
# 2. Clean the data
# Trim whitespace from column names
df.columns = df.columns.str.strip()
```

Removes extra spaces from column names.

STEP 4: Remove Duplicate Rows:

```
# Remove duplicate rows
initial_shape = df.shape
df.drop_duplicates(inplace=True)
print(f"Removed {initial_shape[0] - df.shape[0]} duplicate rows.")
```

Removes extra spaces from column names.

STEP 5: Check for Missing Values

```
# Check for missing values
missing = df.isnull().sum()
print("\nMissing values per column:")
print(missing[missing > 0])
```

Lists columns that contain missing (null) values.

STEP 6: Drop Rows with Missing Values

```
# Drop rows with missing values (or handle them accordingly)
df.dropna(inplace=True)
print(f"\nData shape after dropping missing values: {df.shape}")

# Optional: convert column names to snake_case
df.columns = df.columns.str.lower().str.replace(' ', '_')
```

Removes any rows that have missing values.

Standardizes column names: lowercase + underscores.

STEP 7: Analyze the Data

```
# 4. Analyze the data

# Summary statistics
print("\nSummary Statistics:")
print(df.describe(include='all'))
```

Displays count, mean, std, min, max, and unique values per column.

STEP 8: Value Counts for Categorical Variables

```
# Value counts for categorical variables
for col in df.select_dtypes(include='object').columns:
    print(f"\nValue counts for {col}:")
    print(df[col].value_counts())
```

Loops through categorical columns and shows how often each category appears.

Results & Visualization:

1. Bar Plot—Average Salary by Job Title

Steps:

1. `plt.figure(figsize=(12, 6))`

→ Sets the figure size to make the chart wide and readable.

2. `sns.barplot(...)`

→ Creates a bar plot where:

- `data=df`: Uses the DataFrame `df`.
- `x='job_title'`: Job titles are on the x-axis.
- `y='salary'`: Salaries are on the y-axis.
- `estimator='mean'`: Plots the average salary for each job title.
- `palette='viridis'`: Uses the Viridis color scheme.

3. `plt.xticks(rotation=45)`

→ Rotates x-axis labels 45° for better readability.

4. `plt.title(...)`

→ Adds a title: “Average Salary by Job Title”.

5. `plt.ylabel(...)` / `plt.xlabel(...)`

→ Labels the y-axis as “Average Salary” and the x-axis as “Job Title”.

6. `plt.tight_layout()`

→ Adjusts spacing to prevent overlap of text.

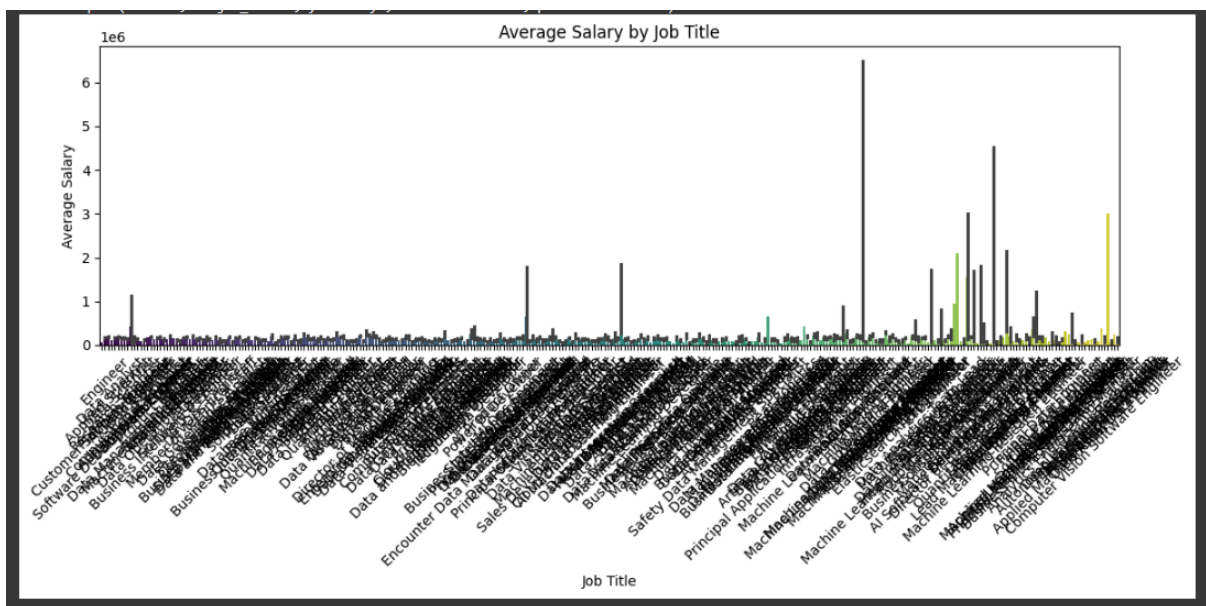
7. `plt.show()`

→ Displays the final plot.

```
#Bar Plot - Average Salary by Job Title

plt.figure(figsize=(12, 6))
sns.barplot(data=df, x='job_title', y='salary', estimator='mean', palette='viridis')
plt.xticks(rotation=45)
plt.title('Average Salary by Job Title')
plt.ylabel('Average Salary')
plt.xlabel('Job Title')
plt.tight_layout()
plt.show()
```

Output:



2. Box Plot—Salary by Experience Level

Steps Involved:

1. Set the Figure Size:

- **Theory:** We define the size of the plot so it appears clear and proportionate.
- **Code:** `plt.figure(figsize=(8, 5))`
- **Explanation:** This creates a figure with a width of 8 inches and a height of 5 inches to ensure a readable plot.

2. Create the Box Plot:

- **Theory:** We use Seaborn's `boxplot` function to visualize the distribution of salary by experience level.
- **Code:** `sns.boxplot(data=df, x='experience_level', y='salary', palette='pastel')`
- **Explanation:**
 - **data=df:** The data is sourced from the `df` DataFrame.
 - **x='experience_level':** This represents the experience levels (e.g., Junior, Mid, Senior) on the x-axis.
 - **y='salary':** This represents the salary values on the y-axis.
 - **palette='pastel':** This specifies a pastel color palette for the plot.

3. Add Title:

- **Theory:** Titles help describe the purpose of the plot, making it clear what the visualization represents.
- **Code:** `plt.title('Salary Distribution by Experience Level')`
- **Explanation:** The title "Salary Distribution by Experience Level" will appear at the top of the plot.

4. Label X-Axis:

- **Theory:** The x-axis label describes the categories along the horizontal axis.
- **Code:** `plt.xlabel('Experience Level')`
- **Explanation:** This sets the label for the x-axis to "Experience Level."

5. Label Y-Axis:

- **Theory:** The y-axis label describes the values along the vertical axis.
- **Code:** `plt.ylabel('Salary')`
- **Explanation:** This sets the label for the y-axis to "Salary."

6. Adjust Layout:

- **Theory:** The `tight_layout()` function automatically adjusts plot elements, ensuring there's no overlap and everything fits well within the figure.
- **Code:** `plt.tight_layout()`
- **Explanation:** This function ensures that titles, labels, and other elements fit properly within the figure area.

7. Display Plot:

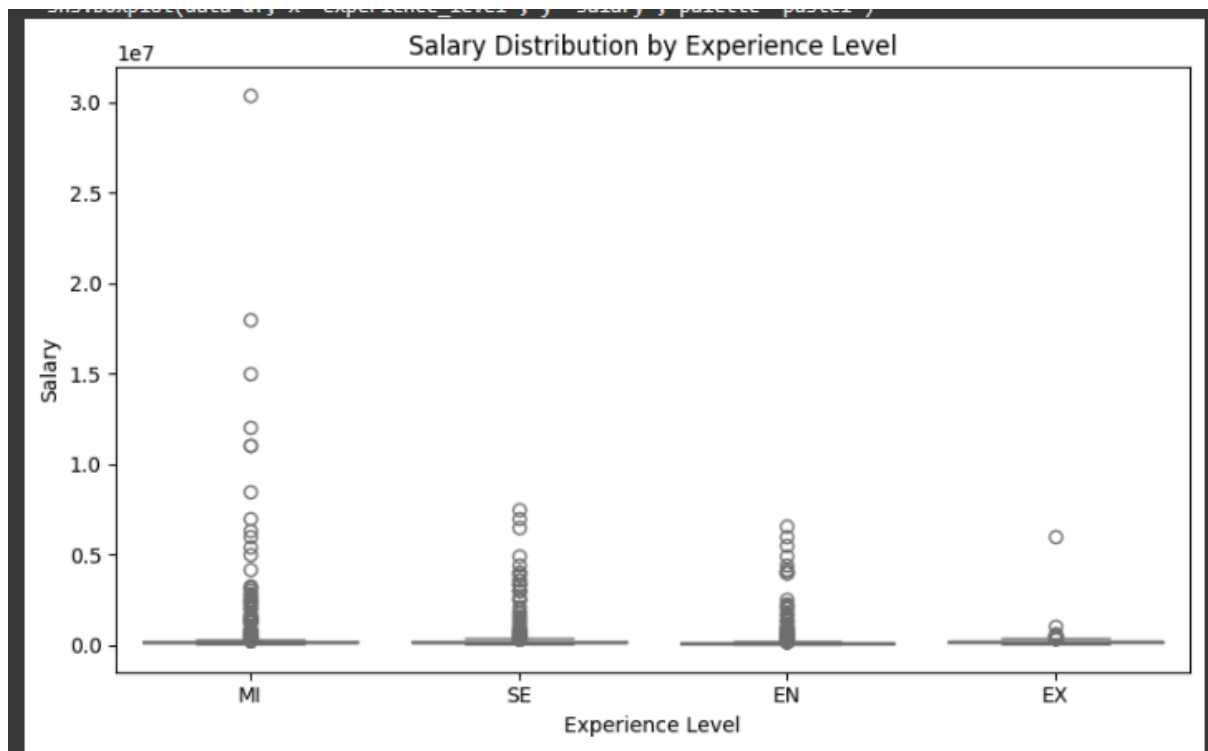
- **Theory:** Finally, we use `plt.show()` to render and display the plot.
- **Code:** `plt.show()`
- **Explanation:** This renders the plot and shows it on the screen.

CODE:

```
#Box Plot - Salary by Experience Level

plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='experience_level', y='salary', palette='pastel')
plt.title('Salary Distribution by Experience Level')
plt.xlabel('Experience Level')
plt.ylabel('Salary')
plt.tight_layout()
plt.show()
```

Output:



3. Pie Chart—Distribution of Company Sizes

Steps Involved:

1. Calculate Company Size Counts:

- **Theory:** The `value_counts()` function calculates the number of occurrences of each unique value in the `company_size` column of the `df` DataFrame.
- **Code:** `company_size_counts = df['company_size'].value_counts()`
- **Explanation:** This line returns a series with the counts of each company size category.

2. Set the Figure Size:

- **Theory:** This step defines the size of the pie chart to make it visually balanced.
- **Code:** `plt.figure(figsize=(6, 6))`
- **Explanation:** A figure of size 6x6 inches is created for the pie chart.

3. Create the Pie Chart:

- **Theory:** The `plt.pie()` function is used to generate the pie chart. The `autopct` argument displays percentages, and `startangle` rotates the chart for better appearance.

Code:

```
#Pie Chart - Distribution of Company Sizes

company_size_counts = df['company_size'].value_counts()
plt.figure(figsize=(6, 6))
plt.pie(company_size_counts, labels=company_size_counts.index, autopct='%1.1f%%', startangle=140, colors=sns.color_palette('Set2'))
plt.title('Company Size Distribution')
plt.tight_layout()
plt.show()
```

- **Explanation:**
 - **company_size_counts:** This provides the data for the pie chart.
 - **labels=company_size_counts.index:** This sets the labels of each section to the unique values of company sizes.
 - **autopct='%1.1f%%':** Displays the percentage of each slice with one decimal point.
 - **startangle=140:** Rotates the pie chart for better appearance.
 - **colors=sns.color_palette('Set2'):** Sets a color palette for the pie chart using Seaborn's **Set2** palette for aesthetics.

Add Title:

- **Theory:** The title provides a brief description of what the pie chart represents.
- **Code:** `plt.title('Company Size Distribution')`
- **Explanation:** Adds the title "Company Size Distribution" to the chart.

Adjust Layout:

- **Theory:** The `tight_layout()` function optimizes the space and ensures that the chart fits neatly within the figure area.

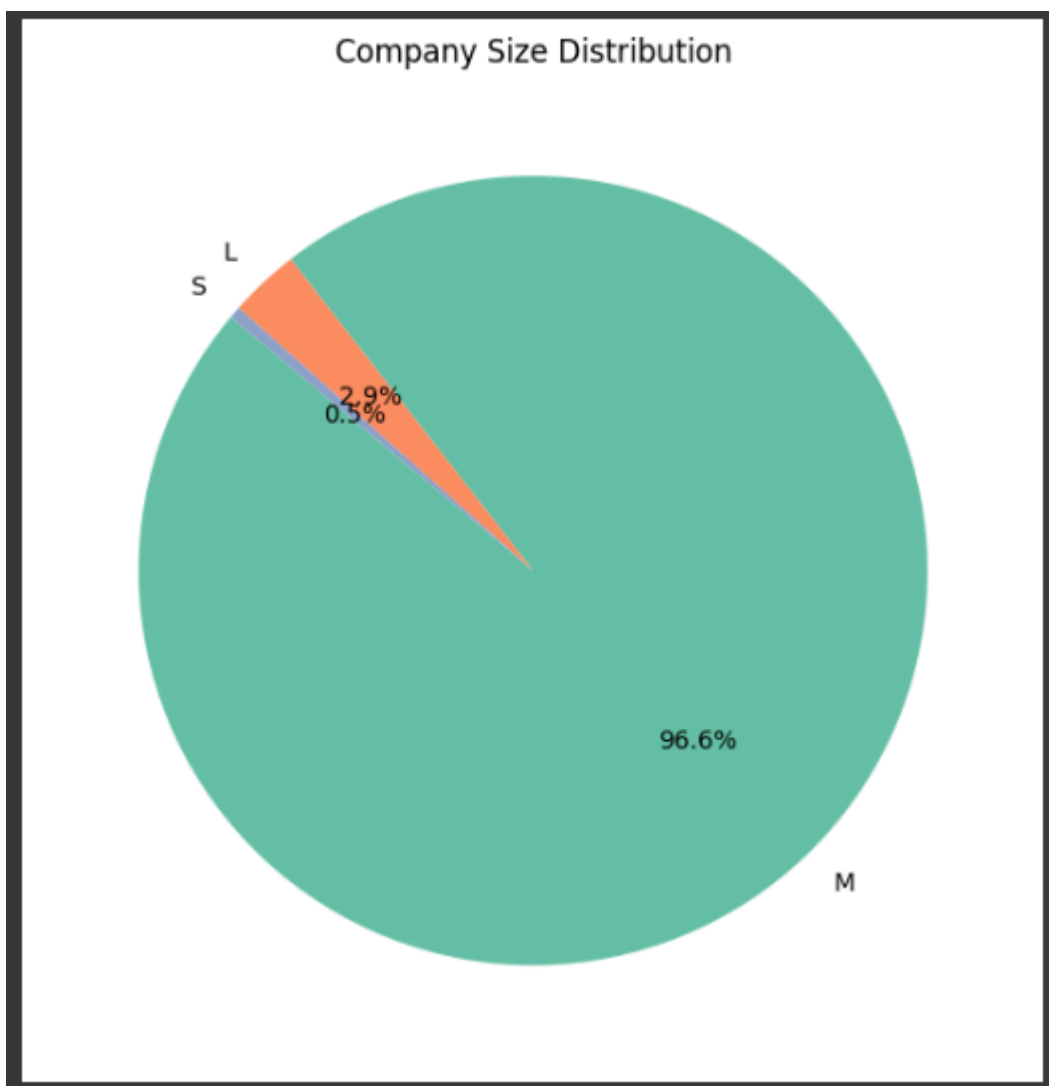
- Code: `plt.tight_layout()`
- Explanation: Adjusts layout to avoid clipping of labels or title.

Display the Plot:

- Theory: Finally, the `plt.show()` function is called to render and display the pie chart.
- Code: `plt.show()`

Explanation: This command will display the pie chart on the screen.

Output:



4. Heatmap—Correlation Between Numeric Columns

Steps Involved:

1. Set Figure Size:

- **Theory:** Defines the size of the figure to ensure the heatmap fits well and is easily readable.
- **Code:** `plt.figure(figsize=(10, 6))`
- **Explanation:** Creates a figure with a width of 10 inches and a height of 6 inches, making the plot large enough to display the heatmap clearly.

2. Generate the Correlation Matrix:

- **Theory:** The `df.corr(numeric_only=True)` method calculates the correlation between the numeric columns in the `df` DataFrame. Correlation measures how strongly two variables are related.
- **Code:** `df.corr(numeric_only=True)`
- **Explanation:** This calculates the correlation matrix of numeric columns only, ensuring the heatmap will represent correlations between numerical features.

3. Create the Heatmap:

- **Theory:** We use `sns.heatmap()` to plot the correlation matrix as a heatmap. The `annot` argument enables annotation of values inside the heatmap, and the `cmap` argument specifies the color palette.
- **Code:**

```
#Heatmap - Correlation Between Numeric Columns

plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

- **Explanation:**

- `df.corr(numeric_only=True)`: The correlation matrix.
- `annot=True`: This annotates the heatmap with the numerical correlation values.
- `cmap='coolwarm'`: Specifies the color palette to represent the values. The coolwarm palette typically uses a color gradient ranging from blue (low correlation) to red (high correlation).
- `fmt='.2f'`: This formats the annotated values to show two decimal places.

Add Title:

- **Theory:** Adding a title makes it clear what the heatmap represents.
- **Code:** `plt.title('Correlation Heatmap')`
- **Explanation:** This adds the title "Correlation Heatmap" at the top of the plot.

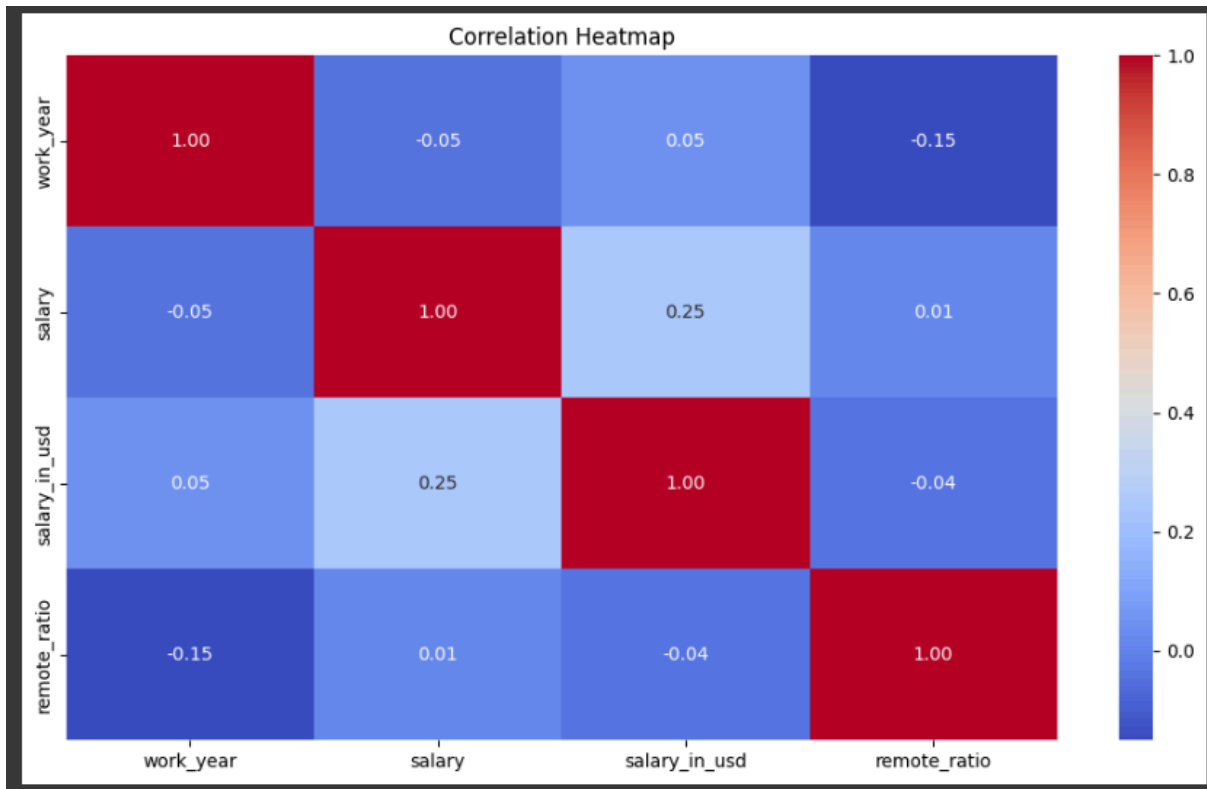
Adjust Layout:

- **Theory:** The `plt.tight_layout()` function automatically adjusts the layout of the plot to ensure everything fits within the figure area without overlapping.
- **Code:** `plt.tight_layout()`
- **Explanation:** Ensures that the plot elements are neatly arranged without clipping.

Display the Plot:

- **Theory:** The `plt.show()` function displays the plot on the screen.
- **Code:** `plt.show()`
- **Explanation:** This renders and shows the heatmap.

Output:



5. Scatter Plot—Salary vs Remote Ratio

Steps Involved:

1. Set Figure Size:

- **Theory:** Defines the size of the figure to ensure the plot is clear and readable.
- **Code:** `plt.figure(figsize=(8, 5))`
- **Explanation:** This creates a figure with a width of 8 inches and a height of 5 inches for a balanced, readable

- scatter plot.

2. Create the Scatter Plot:

- **Theory:** We use `sns.scatterplot()` to create a scatter plot. It plots each point representing a data entry, with the x-axis representing the `remote_ratio` and the y-axis representing salary. The points are colored by experience level.

- i. **Explanation:**

1. `data=df`: The data comes from the `df` DataFrame.
2. `x='remote_ratio'`: The x-axis represents the `remote_ratio` (the percentage of remote work).
3. `y='salary'`: The y-axis represents salary.
4. `hue='experience_level'`: This adds color to the points based on the `experience_level` of the individuals.
5. `palette='Set1'`: Specifies the color palette (`Set1`) for the scatter plot. This palette has distinct colors for different experience levels.

- **Add Title:**

- i. **Theory:** A title makes it clear what the scatter plot represents.
- ii. **Code:** `plt.title('Salary vs Remote Ratio by Experience Level')`

Explanation: This sets the title of the plot to "Salary vs Remote Ratio by Experience Level."

- **Label X-Axis:**
 - i. **Theory:** The x-axis label provides context for the values on the x-axis.
 - ii. **Code:** `plt.xlabel('Remote Ratio (%)')`
 - iii. **Explanation:** This labels the x-axis as "Remote Ratio (%)", indicating that the x-axis shows the percentage of remote work.
- **Label Y-Axis:**
 - i. **Theory:** The y-axis label provides context for the values on the y-axis.
 - ii. **Code:** `plt.ylabel('Salary')`
 - iii. **Explanation:** This labels the y-axis as "Salary," indicating that the y-axis shows salary values.
- **Adjust Layout:**
 - i. **Theory:** The `tight_layout()` function ensures that all the plot elements fit within the figure area without overlapping.
 - ii. **Code:** `plt.tight_layout()`
 - iii. **Explanation:** Automatically adjusts the layout to avoid clipping of labels and title.
- **Display the Plot:**
 - i. **Theory:** Finally, the `plt.show()` function displays the plot.
Code: `plt.show()`

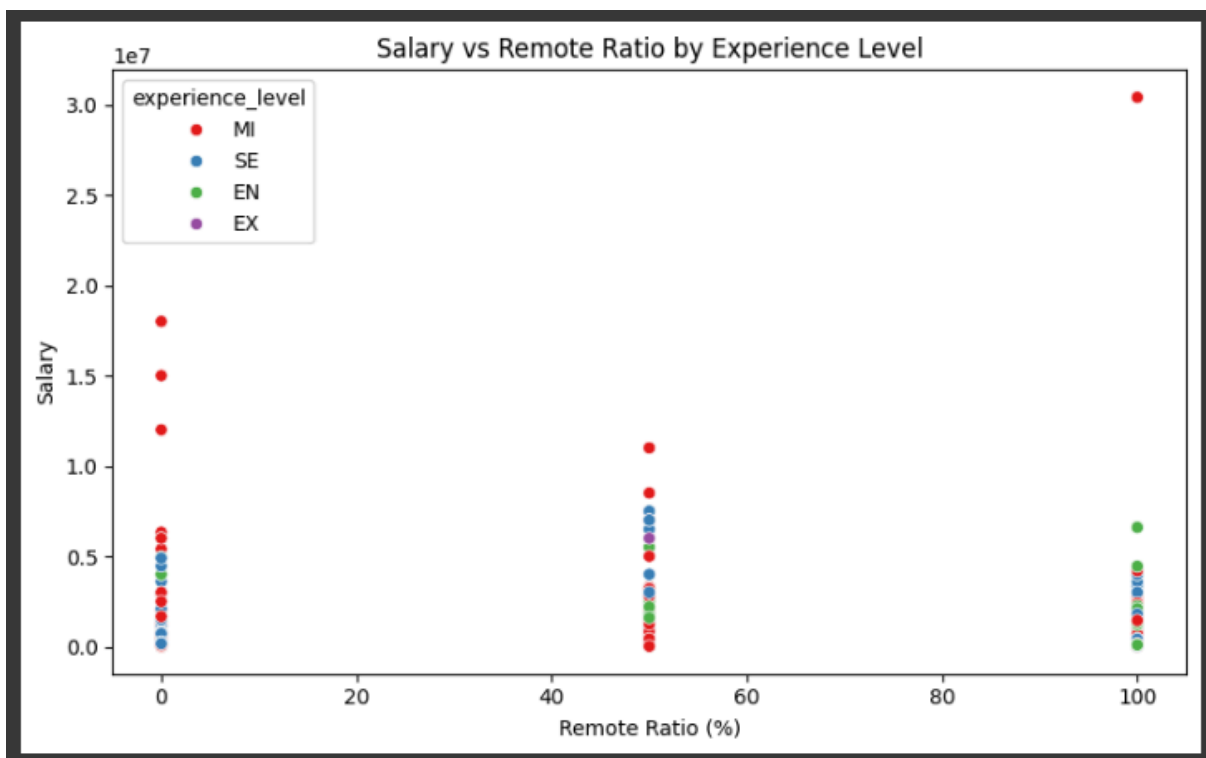
- ii. **Explanation:** This renders and displays the scatter plot on the screen.

CODE:

```
# Scatter Plot - Salary vs Remote Ratio

plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='remote_ratio', y='salary', hue='experience_level', palette='Set1')
plt.title('Salary vs Remote Ratio by Experience Level')
plt.xlabel('Remote Ratio (%)')
plt.ylabel('Salary')
plt.tight_layout()
plt.show()
```

Output:



Conclusion & Future Scope

Future Scope:

1. Predictive Modeling:

Build a machine learning model to predict salary based on job title, experience, company size, remote ratio, etc.

2. Time-Series Analysis (*if data has years*):

Track how salaries have evolved over time for specific roles.

3. Geographical Analysis:

If the dataset includes location info, analyze salary trends by country or region.

4. Salary Normalization:

Normalize salaries across currencies or cost-of-living indexes for international comparison.

5. Interactive Dashboard:

Use tools like Plotly Dash, Tableau, or Power BI to create an interactive salary exploration tool.

Conclusion:

The analysis of the salary dataset provided valuable insights into how various factors influence employee compensation. After thorough data cleaning—removing duplicates and handling missing values—we ensured the dataset was ready for meaningful exploration. The analysis revealed that job title and experience level are significant determinants of salary, with senior and executive-level roles commanding the highest pay. Additionally, company size and remote work ratios also contributed to variations in salary levels. Visualizations such as bar plots, boxplots, and heatmaps highlighted key trends and correlations, allowing for a deeper understanding of workforce dynamics. The salary distribution was skewed, indicating a concentration of salaries at the lower end with a few high-paying outliers. This suggests that while high salaries exist, they are not the norm. Overall, the insights gained can be useful for job seekers, employers, and analysts in understanding compensation trends, planning career moves, and making informed business or HR-related decisions.
